Laboratorio 2 — MSG Foundation

Parte A: decisiones de diseño (D1 a D6)

Curso: Ingeniería de Software I (603501) — Unillanos

Equipo: 8

Rol	Nombre
Integrador	Joseph Sebastian Martinez Forero
Formato y estilo	Looana Marinella Leon Guevara
Vista	Derlys Xiomara Leal Castillo
Controlador	Juan Sebastian Hernandez Ortiz

D1. ¿Es FundsController el Controller de GRASP?

Rol: Controlador

Pregunta: FundsController se llama «controlador», pero ¿es el Controller de GRASP?

Lo que escribe el equipo: *Quién cumple ese papel y por qué importa la diferencia.*

En el laboratorio tenemos dos cosas llamadas «controlador», pero hacen trabajos totalmente distintos. Por un lado está FundsController, que es puro JavaFX: actúa como un adaptador de la infraestructura que solo maneja etiquetas y eventos @FXML. No calcula nada; solo traduce. Por ejemplo, transforma el clic de un botón en una llamada a execute() y formatea el resultado a $ 1160.00. Por otro lado está el Controller en el sentido de GRASP, que en este caso es EstimateFundsForWeek. Este es el verdadero caso de uso de la aplicación: recibe la orden y coordina los repositorios de hipotecas, inversiones y gastos, sin tener idea de que existe una interfaz gráfica.

Esta separación es clave. Si pusiéramos la lógica dentro de FundsController, las fórmulas quedarían atrapadas en JavaFX. No podríamos reutilizarlas en una consola, sería un dolor de cabeza probarlas sin abrir la ventana, y cualquier cambio visual nos obligaría a tocar la lógica de negocio. Eso es exactamente el error que comete el método de la Parte D.

D2. Inyección del caso de uso en FundsController

Decisión (Integrador): FundsController recibe el caso de uso por su constructor, desde App.

Pregunta: ¿qué se pierde si el controlador hiciera new EstimateFundsForWeek(...) con sus propios repositorios?

Si el controlador hiciera new EstimateFundsForWeek(...), tendría que crear también InMemoryMortgageRepository, InMemoryInvestmentRepository e InMemoryOperatingExpensesRepository, porque el constructor del caso de uso exige los tres puertos. Esto tiene tres consecuencias concretas:

1. **Se pierden los datos.** Las hipotecas M-001 y M-002, las inversiones I-01 e I-02 y los gastos de 39000.00 se cargan en la raíz de composición, no en el controlador. Un controlador con repositorios nuevos estaría vacío y mostraría $ 0.00 en lugar de $ 1160.00, salvo que repitiera la carga de datos de Main.
2. **La capa de la interfaz queda atada a las implementaciones.** FundsController importaría clases de infrastructure (los repositorios en memoria) en lugar de depender solo del caso de uso. Cambiar la persistencia obligaría a editar el controlador.
3. **Se pierde la raíz de composición.** Hoy App (igual que Main) es el único lugar que conoce todas las capas y las conecta. Si el controlador también arma dependencias, hay dos lugares que deciden con qué datos funciona el sistema.

Recibir el caso de uso por el constructor, desde App, deja al controlador sin new de casos de uso ni de repositorios, y permite que la ventana y la consola usen el mismo caso de uso.

D3. Ubicación de MoneyFormat

Rol: Formato y estilo

Pregunta: MoneyFormat va en infrastructure.ui y no en domain. ¿Qué tiene que ver el formato $ 1160.00 con la ventana y con las reglas del negocio?

Lo que escribe el equipo: *Qué tiene que ver el formato con la ventana y con las reglas del negocio.*

- El formato monetario $ 1160.00 corresponde a la presentación visual de los valores dentro de la interfaz gráfica, no a las reglas del negocio. El dominio debe trabajar con valores numéricos y reglas propias del negocio, sin depender de cómo se muestran los datos al usuario.
- Por esta razón, la clase MoneyFormat se ubica en infrastructure.ui, ya que su responsabilidad es transformar un valor numérico en una cadena de texto legible para la interfaz. Esta separación permite modificar el formato visual sin afectar los cálculos ni las reglas del negocio, manteniendo una arquitectura organizada y con responsabilidades independientes.

D4. Nombres acordados del contrato

Rol: Vista

Pregunta: Los nombres del contrato (fx:id, onAction) se acuerdan antes de escribir la vista. ¿Qué pasaría si la vista y el controlador los eligieran cada uno por su cuenta?

Lo que escribe el equipo: *Qué pasaría si la vista y el controlador los eligieran cada uno por su cuenta.*

Si el controlador y la vista eligieran sus nombres cada uno por su cuenta la información no coincidiría, ya que el fx:id="totalLabel" no sería igual al campo totalLabel del controlador y el onAction="#onEstimate" del botón no sería igual al método onEstimate(), donde el fx:controller no apuntaría a la clase FundsController real. Java no avisa de esto al compilar, porque el fx:id="totalLabel" es texto aparte. El error aparecería al abrir la ventana y como cada integrante trabaja en su rama, se descubriría tarde, para evitar esto se acuerda los nombres antes del contrato.

D5. Orden de fusión de los cinco PR

Decisión (Integrador): definir el orden de fusión de los PR del equipo.

Pregunta: ¿en qué orden se fusionan los cinco PR y qué dependencia lo obliga?

Orden: 1) feature/javafx-setup, 2) feature/format, 3) feature/view y feature/controller (en cualquier orden entre ellos), 4) feature/app.

- **javafx-setup primero:** todo archivo que importa javafx... solo compila cuando las dependencias de JavaFX están en el pom.xml. FundsController (con @FXML y Label) y App no compilan sin ese PR.
- **format antes que controller:** FundsController llama a MoneyFormat.format(double) para mostrar $ 1160.00. Sin MoneyFormat en main, el controlador no compila.
- **view y controller en cualquier orden:** entre ellos no hay dependencia de compilación. La vista solo necesita que el controlador exista al cargarla, y eso se comprueba en la Parte C.
- **app al final:** App carga funds-view.fxml (Vista) y construye new FundsController(estimateFunds) (Controlador), así que necesita ambos ya en main.

D6. Predicción del equipo

Pregunta: ¿qué archivos de domain y de application van a cambiar?

Ninguno. Lista predicha: vacía. Los cuatro aportes nuevos viven en “infrastructure/ui”, en resources y en la raíz del paquete, y el caso de uso se recibe tal cual.